

Error Taxonomy-Guided Prompt Optimization

Mayank Singh¹ Vikas Yadav² Eduardo Blanco¹

Abstract

Automatic Prompt Optimization (APO) is a powerful approach for extracting performance from large language models without modifying their weights. Many existing methods rely on trial-and-error, testing different prompts or in-context examples until a good configuration emerges, often consuming substantial compute. Recently, natural language feedback derived from execution logs has shown promise as a way to identify how prompts can be improved. However, most prior approaches operate in a bottom-up manner, iteratively adjusting the prompt based on feedback from individual problems, which can cause them to lose the global perspective. In this work, we propose Error Taxonomy-Guided Prompt Optimization (ETGPO), a prompt optimization algorithm that adopts a top-down approach. ETGPO focuses on the global failure landscape by collecting model errors, categorizing them into a taxonomy, and augmenting the prompt with guidance targeting the most frequent failure modes. Across multiple benchmarks spanning mathematics, question answering, and logical reasoning, ETGPO achieves accuracy that is comparable to or better than state-of-the-art methods, while requiring roughly one third of the optimization-phase token usage and evaluation budget.

1. Introduction

Large language models (LLMs) are sensitive to prompt wording, structure, and examples, making prompt design an effective mechanism for improving task performance without modifying model weights. However, manual prompt design requires expertise, trial-and-error, and effort. This has motivated a growing body of work on Automatic Prompt Optimization (APO), which treats prompt design as an optimization problem over discrete natural-language strings. Early approaches include APE (Zhou et al., 2023), which

¹University of Arizona ²ServiceNow. Correspondence to: Mayank Singh <mayanks43@arizona.edu>.

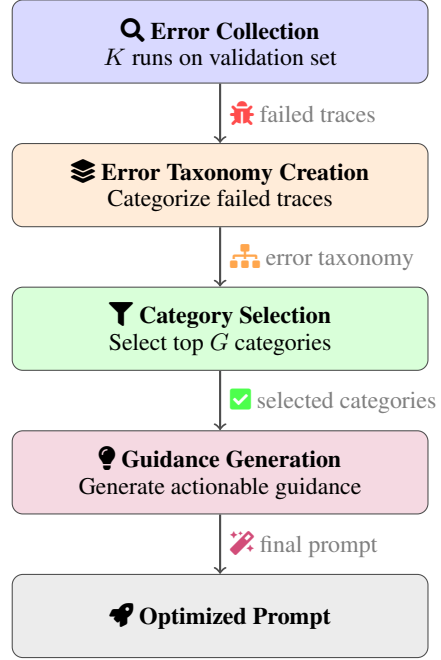


Figure 1. Overview of ETGPO. We first collect errors from failed traces and then organize them into a taxonomy. After filtering to the most prevalent error categories, we generate actionable guidance for each and combine them into the final prompt. This entire process is automated end-to-end.

framed prompt design as a black-box optimization problem; OPRO (Yang et al., 2024), which showed that LLMs can optimize prompts by learning from past attempts; and ProTeGi (Pryzant et al., 2023), which introduced feedback-driven refinement using natural language critiques.

Since then, diverse approaches have emerged: evolutionary methods that evolve prompts through mutation and combination (Guo et al., 2024; Fernando et al., 2024), history-informed methods that use Bayesian optimization to guide search (Opsahl-Ong et al., 2024), and further feedback-driven methods that refine prompts based on execution errors (Yuksekgonul et al., 2024; Agarwal et al., 2025; Agrawal et al., 2025).

We focus on feedback-driven approaches, which have shown strong empirical gains but operate bottom-up: they collect errors on small batches and immediately edit the prompt, risking overfitting to specific examples. To mitigate this,

they allocate substantial budget to repeatedly evaluating each candidate on a validation set (Agrawal et al., 2025).

To address these issues, we propose **Error Taxonomy-Guided Prompt Optimization (ETGPO)**, a top-down APO method that explicitly models the dataset-level error landscape through an error taxonomy. The taxonomy organizes errors found in the execution traces of a base prompt into error categories, along with their prevalence statistics, providing an interpretable view of the errors the LLM makes on the domain of interest. This structure allows us to generate targeted guidance only for the most prevalent error categories. By filtering out long-tail errors, we ensure that prompt edits are more likely to generalize beyond specific problems, reducing the need for iterative validation and improving overall efficiency.

In summary, our contributions are ¹:

1. We propose ETGPO, a top-down approach to APO that builds a global error taxonomy and generates guidance for prevalent failure modes. ETGPO is independent of the task and choice of LLM, while also providing an interpretable view of the target LLM’s weaknesses.
2. We show that ETGPO matches or outperforms state-of-the-art methods on benchmarks spanning mathematics, question answering, and logical reasoning.
3. We achieve this at roughly one third of the optimization cost, demonstrating that a simpler approach can replace expensive iterative refinement.

2. Related Work

There exist many APO methods in the literature, and they can be classified in several different ways. For our purposes, we group them into three categories based on how new candidates are proposed: feedback-driven, evolutionary, and history-informed methods, and discuss representative approaches within each category.

Feedback-driven approaches iteratively refine a prompt by analyzing its performance on examples and using that critique to guide edits. ProTeGi (Pryzant et al., 2023) iteratively collects errors on mini-batches of training examples, uses an LLM to summarize what went wrong, and edits the prompt accordingly. It maintains multiple candidates via beam search, selecting the best by validation performance. TextGrad (Yuksekgonul et al., 2024) frames prompt optimization as gradient descent, where an LLM generates critiques of outputs and another LLM edits the prompt to address them. It follows a single optimization trajectory rather than maintaining multiple candidates via beam search. PromptWizard (Agrawal et al., 2025) also uses an LLM to analyze reasoning failures, identify prompt weaknesses, and

improve the prompt. However, unlike TextGrad, it additionally collects in-context examples from successful runs, jointly optimizing the prompt and in-context examples.

Evolutionary methods maintain multiple prompt candidates, evolving them through combination and mutation while discarding weaker candidates. EvoPrompt (Guo et al., 2024) maintains a pool of prompt candidates and uses an LLM to combine or rewrite them, filtering to top-k after each round. PromptBreeder (Fernando et al., 2024) extends EvoPrompt by evolving both the task prompts and the prompts used to mutate them, so the system learns better mutation strategies over time. GEPA (Agrawal et al., 2025) is a hybrid method that mutates and combines prompts based on execution feedback while maintaining a diverse population where each candidate excels on a different subset of examples, with the best prompt selected by overall coverage.

History-informed methods improve prompts iteratively, using the trajectory of past attempts and their scores to inform new candidates. OPRO (Yang et al., 2024) uses an LLM to propose new prompts while showing it the history of past prompts and their scores, letting it learn from previous attempts. APE (Zhou et al., 2023) uses an LLM to generate prompt candidates and iteratively refines them by paraphrasing top performers, treating successful prompts as implicit historical guidance. MIPRO (Opsahl-Ong et al., 2024) also uses an LLM to generate prompt candidates, but additionally optimizes in-context examples from successful runs, using Bayesian optimization to select promising combinations.

ETGPO, our method, is feedback-driven but takes a different approach than existing methods. Rather than maintaining multiple prompt candidates like evolutionary and history-informed methods, or iteratively editing based on individual errors like other feedback-driven methods, ETGPO builds a global error taxonomy and generates targeted guidance for the most prevalent failure modes. This top-down approach enables surgical, high-impact interventions, reducing the need for repeated candidate evaluations and lowering overall optimization cost.

3. Method

Before describing our method, we introduce the problem setting and some necessary definitions.

We consider a dataset of problems, each with a ground-truth answer. We are also given a base system prompt, which is used to query a large language model together with the problem text to produce a predicted answer. The goal of automatic prompt optimization is to improve this base system prompt by appending additional text such that the accuracy of the model’s responses, measured against the ground-truth answers, improves over the dataset under consideration overall.

¹Code at: <https://github.com/mayanks43/etgpo>

We now introduce the definitions. First, we distinguish between two types of LLMs used in our method. The *backbone LLM* is the model used to solve problems in the dataset under consideration (i.e., generate reasoning traces and answers). The *optimizer LLM* is the model used to improve the system prompt that is passed to the backbone LLM.

Second, we define *failed traces* as the reasoning traces of problems in the dataset for which the backbone LLM fails to produce the correct answer. Third, we define an *error taxonomy* as a categorization of the errors identified in the failed traces, along with their prevalence statistics. Fourth, we define *guidance* as a set of instructional text designed to prevent the kinds of errors described by the error taxonomy.

Next, we describe ETGPO, our method. It consists of four steps: (1) error collection, (2) error taxonomy creation, (3) error category selection, and (4) guidance generation. We describe each step in detail below:

3.1. Error Collection

In the first step, we take the validation set of the dataset at hand together with the base system prompt and query the backbone LLM on this set of problems. We then collect the failed traces, along with the problem statements, correct answers, and predicted answers. Rather than making a single pass over the validation set, we perform K runs to capture the variety of errors that can arise from the same problem due to the LLM’s stochastic nature.

3.2. Error Taxonomy Creation

We next create a taxonomy of the errors found in the failed traces. For each failed trace, we prompt the optimizer LLM to analyze the problem, given the problem statement, the correct answer, the model’s reasoning trace, and the model’s predicted answer. The optimizer LLM is asked to identify where the reasoning first goes wrong, determine the nature of the error, and explain why this error led to an incorrect answer. Based on this analysis, the LLM assigns the trace to an error category.

In practice, the number of failed traces can be large, which risks exceeding the context length of the optimizer LLM. To address this, we perform taxonomy creation in a batched manner. In the first call, we provide the optimizer LLM with a batch of B failed traces, along with their associated information as described above, and ask it to group them into categories based on the error analysis. In response, the LLM assigns each trace to a newly created error category and produces auxiliary information for each category, including a summary, a description, an example, the error type, and an explanation of why the error leads to an incorrect answer. Each error category is required to be self-contained and understandable without reference to the original problems, so

Algorithm 1 Error Taxonomy-Guided Prompt Optimization

Input: Validation set V , base prompt P , backbone LLM, optimizer LLM

Parameters: K (collection runs), B (batch size), G (max error categories)

Output: Optimized prompt P^*

// Step 1: Error Collection

$F \leftarrow \emptyset$

for $k = 1$ **to** K **do**

for each problem in V **do**

 Query backbone LLM with P and problem

if prediction is incorrect **then**

 Add (problem, trace, prediction) to F

end if

end for

end for

// Step 2: Error Taxonomy Creation

$C \leftarrow \emptyset$

for each batch of B traces in F **do**

 Use optimizer LLM to identify error categories

 Update C with new or existing error categories

end for

// Step 3: Error Category Selection

Filter C to error categories with ≥ 2 problems

Sort C by failure count (descending)

$S \leftarrow$ top G error categories from C

// Step 4: Guidance Generation

Use optimizer LLM to generate guidance for S

$P^* \leftarrow P +$ preamble + guidance texts

return P^*

that the resulting guidance generalizes beyond the specific examples encountered during optimization.

In subsequent calls, we repeat this process with the additional constraint that the LLM should reuse existing error categories whenever possible. Accordingly, in these calls, we also provide the LLM with the current set of error categories, along with metadata such as the number of traces assigned to each category so far. At the end of this process, we obtain a set of error categories that form the error taxonomy, together with their prevalence statistics. The full prompt for this step is provided in Section B.

3.3. Error Category Selection

After building the error taxonomy, we select a subset of error categories for generating guidance that helps the backbone LLM avoid such errors. Using all error categories is impractical for several reasons: it would result in an overly

Original System Prompt

Please think step by step and then solve the task.

System Prompt Generated by ETGPO

Please think step by step and then solve the task.

To maximize accuracy on challenging math problems, carefully follow these targeted instructions for each common error type. Each guidance item below addresses a specific failure pattern and provides actionable steps and examples to help you avoid systematic mistakes.

1. Algebraic or Geometric Calculation Error

Description: This error occurs when you make mistakes in algebraic manipulations, geometric calculations, or applying formulas, even after setting up the correct equations or relationships. These mistakes can include sign errors, incorrect formula usage, or miscalculating distances or intersections.

Actionable Advice:

- Double-check every algebraic manipulation and calculation step, especially in multi-step derivations.
- Explicitly write out each step, and verify intermediate results before proceeding.
- For geometric problems, use coordinates or explicit constructions where possible, and check all distances and relationships carefully.
- After obtaining a result, substitute it back into the original equations or geometric setup to verify correctness.

WRONG Example: You are given an equilateral triangle inscribed in a circle, and certain tangent circles. You miscalculate the distances between centers or intersection points, leading to an answer of 16 instead of the correct 378.

CORRECT Example: You carefully assign coordinates to all relevant points, write out the equations for the circles and lines, and check all calculations step by step. After finding the side length, you substitute it back into the geometric relationships to confirm it fits all constraints, arriving at the correct answer of 378.

2. Incorrect Generalization from Patterns or Heuristics

Description: This error happens when you assume a general pattern or heuristic applies based on limited data or intuition, without rigorous justification. You may extrapolate from a few computed terms or from a perceived similarity to a well-known sequence, leading to an incorrect generalization.

Actionable Advice:

- Do not assume a pattern holds unless you can rigorously prove it for all relevant cases.
- Avoid drawing conclusions from a small number of computed terms; instead, analyze the underlying structure or derive a general formula.
- If you suspect a pattern, attempt to prove it by induction or by analyzing the problem’s constraints.
- Always check for exceptions or edge cases that might break the pattern.

[... additional guidance items omitted for brevity ...]

Figure 2. Comparison between the original system prompt and the ETGPO-optimized prompt for AIME, using GPT-4.1-mini as the backbone model and GPT-4.1 as the optimizer model. The complete optimized prompt has 10 blocks, of which only the first block and a portion of the second are shown.

long prompt, and many categories are rare or occur in only a single problem. Focusing on the most common errors is likely to have greater impact.

For these reasons, we focus on selecting the most important error categories. For each error category, we obtain two types of statistics from the previous step: the *problem count*, defined as the number of unique problems exhibiting the error, and the *failure count*, defined as the number of failed traces associated with the category. To avoid optimizing for error categories that are overly specific to individual problems, we filter out categories associated with only a single problem. After this filtering step, we select the top G error categories by failure count for guidance generation in the next step.

3.4. Guidance Generation

We next generate actionable guidance text for each selected error category. We prompt the optimizer LLM with information about each category, including its description, an example, and an explanation of why it leads to an incorrect answer, and ask it to produce guidance text for each category. The LLM is tasked with generating actionable advice together with concrete incorrect and correct examples. In addition, the LLM generates a preamble that introduces the guidance. All guidance texts for the selected error categories, along with the preamble, are generated in a single LLM call. After receiving the response from the LLM, the preamble and guidance texts are appended to the original system prompt. The output of this step is the final improved system prompt. The full prompt used to query the optimizer LLM in this step is provided in Section C. Figure 2 shows an example prompt generated by our method. Additional ETGPO-generated prompts are provided in Section D.

Table 1. Results (mean accuracy with confidence intervals across all runs) obtained using basic chain-of-thought and three automatic prompt optimization methods, with **GPT-4.1-mini** used as the backbone model and GPT-4.1 used as the optimizer. ETGPO (our method) outperforms prior methods on average and on all but one benchmark. Note: **Bold** = best; underline = second-best.

Method	Math Reasoning		General	Multi-hop	Logical Reasoning		Avg.
	AIME	HMMT			AR-LSAT	FOLIO	
Chain of Thought	47.08 $\pm$ 1.43	47.14 $\pm$ 1.23	78.82 $\pm$ 0.30	74.25 $\pm$ 0.63	74.35 $\pm$ 0.91	75.09 $\pm$ 0.67	66.12
MIPROv2	47.66 $\pm$ 1.22	45.89 $\pm$ 1.31	78.37 $\pm$ 0.42	75.20 $\pm$ 0.60	<u>76.36 $\pm$ 1.11</u>	<u>79.71 $\pm$ 0.92</u>	67.20
GEPA	49.06 $\pm$ 1.51	47.86 $\pm$ 1.38	79.55 $\pm$ 0.58	76.35 $\pm$ 0.51	75.60 $\pm$ 0.98	77.83 $\pm$ 0.90	<u>67.71</u>
ETGPO (Ours)	49.06 $\pm$ 1.36	50.47 $\pm$ 1.28	<u>79.40 $\pm$ 0.66</u>	77.30 $\pm$ 0.45	76.93 $\pm$ 0.76	81.34 $\pm$ 0.60	69.08

Table 2. Optimization-phase token usage (total input plus output tokens, truncated to thousands) for three automatic prompt optimization methods, with **GPT-4.1-mini** used as the backbone model and GPT-4.1 used as the optimizer. The Δ column denotes the ratio of each method’s average token usage to that of ETGPO. ETGPO (our method) consistently uses the fewest tokens across all benchmarks. Note: **Bold** = lowest; underline = second-lowest.

Method	AIME	HMMT	MMLU-Pro	Musique	AR-LSAT	FOLIO	Avg.	Δ
MIPROv2	14,332	11,629	3,859	23,735	6,954	2,275	10,464	4.6 $\times$
GEPA	<u>12,495</u>	<u>10,249</u>	<u>2,623</u>	<u>6,093</u>	<u>5,144</u>	<u>2,320</u>	<u>6,487</u>	2.8 $\times$
ETGPO (Ours)	4,377	4,025	595	2,849	1,372	453	2,279	—

We experiment with two styles of guidance: a short style, consisting of one to two sentences per error category, and a detailed style, which includes a full description, actionable advice, and incorrect and correct examples. We later perform an ablation study comparing these two styles.

4. Evaluation

In this section, we discuss the evaluation setup and results for our method on various datasets.

4.1. Datasets

We evaluate our method on six datasets spanning multiple domains: two mathematics datasets (AIME, HMMT), general question answering (MMLU-Pro), multi-hop reasoning (MuSiQue), and logical and analytical reasoning (AR-LSAT, FOLIO). We describe them below:

AIME (Art of Problem Solving): The American Invitational Mathematics Examination (AIME) is a challenging high-school mathematics competition that tests advanced mathematical skills. We use questions from AIME to evaluate the mathematical reasoning abilities of LLMs. In our experiments, we use 90 problems from the 2022–2024 editions as the validation set and 30 problems from the 2025 edition as the test set.

HMMT (Harvard-MIT Mathematics Tournament): The Harvard-MIT Mathematics Tournament (HMMT) is another challenging high-school mathematics competition. For our experiments, we use 90 problems from the February 2023, 2024, and 2025 editions as the validation set, and 30

problems from the November 2025 edition as the test set. We use the publicly available datasets released by Math-Arena (Balunović et al., 2026).

MMLU-Pro (Wang et al., 2024): MMLU-Pro is a more challenging variant of the MMLU benchmark (Hendrycks et al., 2021), a general-domain question answering dataset. Compared to the original benchmark, which provides four answer choices per question, MMLU-Pro includes ten options per question. Since the original validation set contains only 70 problems, we instead create larger validation and test sets by partitioning the full test set into two subsets. We then sample 100 problems from the validation subset and 500 problems from the test subset to form the validation and test sets, respectively.

MuSiQue (Trivedi et al., 2022): MuSiQue is a multi-hop reasoning dataset whose questions require answering multiple implicit subquestions to arrive at the correct answer. The dataset is explicitly constructed to avoid questions that can be solved by answering only a single subquestion, thereby requiring genuine multi-hop reasoning. Due to the large size of the dataset, we randomly sample 160 questions from the training split to form the validation set and 500 questions from the validation split to form the test set.

AR-LSAT (Zhong et al., 2021): AR-LSAT is an analytical reasoning dataset derived from questions in the Law School Admission Test (LSAT). We use a random subset of 100 questions from the validation split as the validation set and include all 230 questions from the test split as the test set.

FOLIO (Han et al., 2024): FOLIO is a dataset designed to test complex logical reasoning abilities. Each instance

Table 3. Results (mean accuracy with confidence intervals across all runs) obtained using basic chain-of-thought and three automatic prompt optimization methods, with **DeepSeek-V3.1** used as the backbone model and GPT-4.1 used as the optimizer. ETGPO (our method) outperforms prior methods on average and on three of six benchmarks. Note: **Bold** = best; underline = second-best.

Method	Math Reasoning		General	Multi-hop	Logical Reasoning		Avg.
	AIME	HMMT			AR-LSAT	FOLIO	
Chain of Thought	65.52 $\pm$ 1.25	53.65 $\pm$ 1.28	82.74 $\pm$ 0.53	76.92 $\pm$ 0.39	90.65 $\pm$ 0.61	75.58 $\pm$ 0.88	74.18
MIPROv2	<u>68.23 $\pm$ 1.25</u>	60.68 $\pm$ 1.51	83.73 $\pm$ 0.54	76.80 $\pm$ 0.61	91.66 $\pm$ 0.68	<u>82.33 $\pm$ 0.57</u>	<u>77.24</u>
GEPA	64.48 $\pm$ 1.33	57.55 $\pm$ 1.30	83.76 $\pm$ 0.66	76.38 $\pm$ 0.55	90.92 $\pm$ 0.57	82.45 $\pm$ 0.40	75.92
ETGPO (Ours)	69.74 $\pm$ 1.33	<u>58.65 $\pm$ 1.62</u>	83.65 $\pm$ 0.52	79.03 $\pm$ 0.72	<u>91.44 $\pm$ 0.52</u>	82.45 $\pm$ 0.93	77.49

Table 4. Optimization-phase token usage (total input plus output tokens, truncated to thousands) for three automatic prompt optimization methods, with **DeepSeek-V3.1** used as the backbone model and GPT-4.1 used as the optimizer. The Δ column denotes the ratio of each method’s average token usage to that of ETGPO. ETGPO (our method) consistently uses the fewest tokens across all benchmarks. Note: **Bold** = lowest; underline = second-lowest.

Method	AIME	HMMT	MMLU-Pro	Musique	AR-LSAT	FOLIO	Avg.	Δ
MIPROv2	17,400	17,027	4,785	26,096	10,735	3,276	13,220	6.5 $\times$
GEPA	<u>9,899</u>	<u>9,923</u>	<u>3,300</u>	<u>6,449</u>	<u>5,523</u>	<u>2,550</u>	<u>6,274</u>	3.1 $\times$
ETGPO (Ours)	3,098	3,295	778	3,063	1,378	507	2,020	—

consists of premises and a conclusion, and the task is to determine whether the conclusion logically follows from the premises. For our experiments, we use a random subset of 100 questions from the training split as the validation set and all 203 questions from the validation split as the test set.

4.2. Baselines

We include Chain of Thought (Wei et al., 2023) as our primary baseline, since the other APO baselines in this paper improve upon its prompt. We also include two state-of-the-art APO methods for comparison: GEPA (Agrawal et al., 2025) and MIPRO (Opsahl-Ong et al., 2024). We use the DSPy (Khattab et al., 2023) implementations of GEPA and MIPRO; in DSPy, MIPRO is implemented as MIPROv2. For both GEPA and MIPRO, we use the heavy optimization mode, which uses the largest number of evaluations to search for the best prompt. Our goal is to compare against the strongest versions of these methods available.

4.3. Experimental Results

Accuracy results We evaluate our method and the baselines on the test sets of the listed datasets. A subset of the results is shown in Table 1 (backbone LLM: GPT-4.1-mini (OpenAI, 2025b)) and Table 3 (backbone LLM: DeepSeek-V3.1 (DeepSeek, 2025)). All reported numbers correspond to the accuracy achieved by the final prompts produced by each method when evaluated on the corresponding test sets.

For all APO methods reported in these tables, we use GPT-4.1 (OpenAI, 2025a) as the optimizer LLM. To reduce com-

putational cost, we run each APO method only once per dataset, but evaluate the resulting prompt multiple times. For the smaller mathematics datasets, we perform 64 runs per problem. For the larger datasets, we use fewer runs per problem, specifically 8 runs for MuSiQue and MMLU-Pro, and 16 runs for AR-LSAT and FOLIO. We use a high number of runs per prompt to obtain tighter confidence intervals and stronger statistical reliability. Following recent work (Ho et al., 2025), we use LLM-based answer equivalence checking: a prediction is correct if it exactly matches the ground truth or, when exact match fails, is judged equivalent by a strong LLM (GPT-5 (Singh et al., 2025)).

For our method, we use the same hyperparameter values across all datasets, chosen based on a hyperparameter search conducted on the AIME validation set. To reduce computational cost, we do not perform a separate hyperparameter search for each dataset. Specifically, we set K , the number of error collection runs, to 5; G , the number of selected error categories, to 10; and use detailed guidance texts rather than the short variant. We do not perform a search over the batch size parameter B and instead use the default value of 6. Tuning these hyperparameters on a per-dataset basis could further improve performance.

We now discuss the results shown in Table 1 and Table 3. Our method achieves the best average performance across both tables, consistently matching or outperforming other APO methods. On individual datasets, it performs best on 5 out of 6 with GPT-4.1-mini as the backbone LLM, and 3 out of 6 with DeepSeek-V3.1.

In terms of absolute improvements on individual datasets,

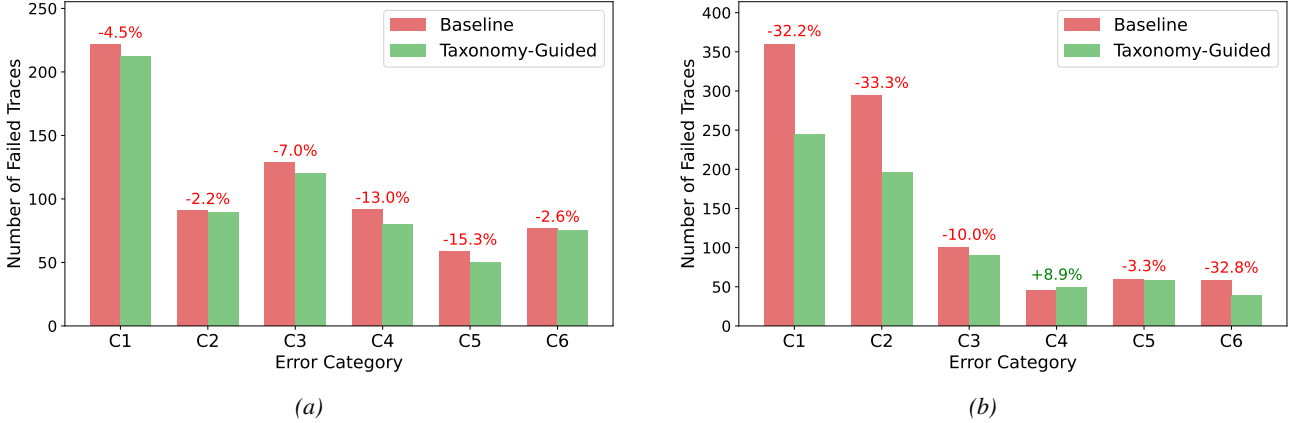


Figure 3. Count of failed traces before and after optimization with ETGPO for the top 6 error categories from the error taxonomy on the validation sets of (a) AIME and (b) FOLIO. All but one category shows fewer failed traces, supporting our hypothesis that targeted guidance reduces error prevalence.

Table 5. Effect of varying K , the number of error collection runs, on final prompt accuracy for the AIME and FOLIO validation sets. Accuracy saturates at different values of K , suggesting the optimal number of runs is dataset-dependent.

K	AIME	FOLIO
Baseline	52.35 $\pm$ 0.91	72.50 $\pm$ 0.08
1	52.61 $\pm$ 0.59	79.70 $\pm$ 0.37
5	53.72 $\pm$ 0.28	79.53 $\pm$ 0.53
10	53.43 $\pm$ 1.06	78.32 $\pm$ 0.38

the largest gains are observed on FOLIO for both backbone models, with improvements of approximately 6–7 percentage points in accuracy. AIME, HMMT, and MuSiQue also show strong gains, ranging from roughly 2 to 5 percentage points across the two backbone models consistently. In contrast, gains on MMLU-Pro are modest, which we attribute to the fact that these questions often require more specific domain knowledge that is less likely to be improved through general guidance alone; retrieval-based approaches may be more suitable in this setting. AR-LSAT shows a strong improvement of approximately 2.5 percentage points when using GPT-4.1-mini as the backbone LLM. However, performance on AR-LSAT appears to be saturated for DeepSeek-V3.1, with accuracies approaching 90%, leaving limited room for further improvement. In this regime, the execution logs provide little additional signal from which to extract useful guidance, resulting in marginal gains.

Token usage results In Table 2 and Table 4, we report the total number of input and output tokens consumed during the optimization phase of each APO method evaluated, including our method, with all values reported in thousands and truncated for clarity. In all cases, the reported token counts correspond to the sum of tokens consumed by the

backbone model and, when applicable, the optimizer LLM.

Across both backbone LLMs and all datasets, our method consistently uses the fewest tokens during the optimization phase. In particular, the total token usage of our method is approximately one third of that of the next most expensive method, GEPA. These results demonstrate that our method achieves performance that is on par with or better than state-of-the-art APO methods, while being substantially more token-efficient during optimization.

4.4. Detailed Analysis

We next present a set of analysis experiments designed to study the impact of various hyperparameters. All experiments are conducted on the validation sets of AIME and FOLIO, using GPT-4.1-mini as the backbone model and GPT-4.1 as the optimizer model. For each configuration, we run each compared method three times, with 20 evaluation runs per generated prompt. Unless otherwise specified, all other hyperparameters are fixed to their default values. We also include the base Chain of Thought prompt as a baseline in these analyses. Note that an experiment with G , which determines the number of top error categories selected for guidance generation, can be found in Section A.

Number of error collection runs In the first experiment, shown in Table 5, we study the effect of the parameter K , which controls the number of error collection runs, on the accuracy of the final prompt. The effect of K varies across datasets. For AIME, accuracy improves as K increases from 1 to 5, but saturates at $K = 10$. For FOLIO, the best performance is achieved already at $K = 1$, with accuracy saturating for larger values. This suggests that additional execution traces provide diminishing returns for FOLIO, likely because multiple runs of the same problem do not yield substantially different error patterns.

Table 6. Ablation study on the components of ETGPO. The first ablation replaces taxonomy construction with raw failure sampling. The second ablation replaces LLM-generated guidance with direct insertion of error category descriptions. The third ablation compares short guidance with detailed guidance containing fuller explanations and examples. Overall, each ablation results in a reduction in average performance, indicating that all components contribute to the effectiveness of ETGPO.

Method	AIME	FOLIO	Avg.	Δ
ETGPO	53.72 ± 0.28	79.53 ± 0.53	66.63	–
Ablation 1: Raw Sampling	53.39 ± 0.84	77.13 ± 0.45	65.26	-1.37
Ablation 2: Direct Categories	52.52 ± 1.27	78.33 ± 0.96	65.43	-1.20
Ablation 3: Short Guidance	52.20 ± 0.96	76.62 ± 0.90	64.41	-2.22

Error reduction In the second experiment, shown in Figure 3, we examine whether the generated guidance effectively reduces the types of errors captured by the different error categories. Using the optimizer LLM for categorization, we compare the number of failed traces assigned to the top six error categories before optimization (baseline) and after optimization, evaluated on the validation sets of AIME and FOLIO, respectively. As shown in the bar charts, the number of failed traces decreases consistently across all error categories after optimization, demonstrating the effectiveness of the prompts generated by our method in mitigating these errors.

4.5. Ablation Experiments

We also conduct a set of ablation experiments to assess the contribution of different components of our method. The results are reported in Table 6. Similar to the analysis experiments, all ablations are conducted on the validation sets of AIME and FOLIO, using GPT-4.1-mini as the backbone model and GPT-4.1 as the optimizer model. For each ablation setting, we run the method three times, with 20 evaluation runs per generated prompt.

Error taxonomy creation The first ablation studies the importance of the error taxonomy creation step. Specifically, it examines whether explicitly categorizing errors across failed traces provides additional value over directly using raw failed traces. In this ablation, we skip taxonomy creation and instead pass a sample of 10 failed traces (one per problem) directly to the guidance generation step. The results indicate that error taxonomy creation is beneficial on average, with performance on FOLIO dropping by approximately 2 percentage points when this step is removed. In contrast, the drop isn’t as significant on AIME, suggesting that a small sample of failed traces might be sufficient to cover the most salient error patterns for this dataset.

Guidance generation The second ablation evaluates the role of the guidance generation step itself. Rather than generating guidance text, this ablation directly constructs the final prompt by inserting the selected error category descriptions as bullet points, preceded by a generic preamble

encouraging the model to watch for common error patterns. The results show a clear performance degradation across both datasets, indicating that explicit guidance generation adds value beyond error category descriptions alone. In particular, the guidance generation step contributes actionable advice and concrete correct and incorrect examples that help the model better understand when and how each error category applies.

Detailed vs short guidance The third ablation examines the effect of guidance granularity. We test a variant of the final prompt in which each error category is summarized using only one to two lines, rather than a more detailed description. As shown in the results, this ablation yields the largest performance drop across both datasets, with an average decrease of approximately two percentage points. This finding highlights the importance of providing descriptive guidance for each error category in the final prompt.

5. Conclusion

In this work, we proposed **Error Taxonomy-Guided Prompt Optimization (ETGPO)**, a feedback-driven APO method that builds an error taxonomy to provide an interpretable view of the dataset-level error landscape and generates guidance only for the most prevalent error categories. By focusing on highly prevalent errors, we ensured that prompt edits generalize beyond the validation set. This allowed us to bypass expensive iterative validation, making our method substantially more efficient, as demonstrated in Section 4. Despite this efficiency, ETGPO remained highly competitive with state-of-the-art methods, outperforming on multiple datasets across mathematical reasoning, question answering, and logical reasoning.

References

Agarwal, E., Magazine, R., Singh, J., Dani, V., Ganu, T., and Nambi, A. PromptWizard: Optimizing prompts via task-aware, feedback-driven self-evolution. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Findings of the Association for Computational Linguistics*.

- tics: *ACL 2025*, pp. 19974–20003, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi: 10.18653/v1/2025.findings-acl.1025. URL <https://aclanthology.org/2025.findings-acl.1025/>.
- Agrawal, L. A., Tan, S., Soylu, D., Ziems, N., Khare, R., Opsahl-Ong, K., Singhvi, A., Shandilya, H., Ryan, M. J., Jiang, M., Potts, C., Sen, K., Dimakis, A. G., Stoica, I., Klein, D., Zaharia, M., and Khattab, O. Gepa: Reflective prompt evolution can outperform reinforcement learning, 2025. URL <https://arxiv.org/abs/2507.19457>.
- Art of Problem Solving. Aime problems and solutions. URL https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions. Accessed: 2025-01-27.
- Balunović, M., Dekoninck, J., Petrov, I., Jovanović, N., and Vechev, M. Matharena: Evaluating llms on uncontaminated math competitions, 2026. URL <https://arxiv.org/abs/2505.23281>.
- DeepSeek. Deepseek-v3.1 release, 2025. URL <https://api-docs.deepseek.com/news/news250821>. Accessed: 2025-01-27.
- Fernando, C., Banarse, D. S., Michalewski, H., Osindero, S., and Rocktäschel, T. Promptbreeder: Self-referential self-improvement via prompt evolution. In Salakhutdinov, R., Kolter, Z., Heller, K., Weller, A., Oliver, N., Scarlett, J., and Berkenkamp, F. (eds.), *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 13481–13544. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/fernando24a.html>.
- Guo, Q., Wang, R., Guo, J., Li, B., Song, K., Tan, X., Liu, G., Bian, J., and Yang, Y. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=ZG3RaNIso8>.
- Han, S., Schoelkopf, H., Zhao, Y., Qi, Z., Riddell, M., Zhou, W., Coady, J., Peng, D., Qiao, Y., Benson, L., Sun, L., Wardle-Solano, A., Szabo, H., Zubova, E., Burtell, M., Fan, J., Liu, Y., Wong, B., Sailor, M., Ni, A., Nan, L., Kasai, J., Yu, T., Zhang, R., Fabbri, A. R., Kryscinski, W., Yavuz, S., Liu, Y., Lin, X. V., Joty, S., Zhou, Y., Xiong, C., Ying, R., Cohan, A., and Radev, D. Folio: Natural language reasoning with first-order logic, 2024. URL <https://arxiv.org/abs/2209.00840>.
- Harvard-MIT Mathematics Tournament. Hmmt archive of problems and results. URL <https://www.hmmt.org/www/archive/problems>. Accessed: 2025-01-27.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding, 2021. URL <https://arxiv.org/abs/2009.03300>.
- Ho, X., Huang, J., Boudin, F., and Aizawa, A. Llm-as-a-judge: Reassessing the performance of llms in extractive qa, 2025. URL <https://arxiv.org/abs/2504.11972>.
- Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T. T., Moazam, H., Miller, H., Zaharia, M., and Potts, C. Dspy: Compiling declarative language model calls into self-improving pipelines, 2023. URL <https://arxiv.org/abs/2310.03714>.
- OpenAI. Introducing gpt-4.1 in the api, 2025a. URL <https://openai.com/index/gpt-4-1/>. Accessed: 2025-01-27.
- OpenAI. Gpt-4.1 mini model — openai api, 2025b. URL <https://platform.openai.com/docs/models/gpt-4.1-mini>. Accessed: 2025-01-27.
- Opsahl-Ong, K., Ryan, M. J., Purtell, J., Broman, D., Potts, C., Zaharia, M., and Khattab, O. Optimizing instructions and demonstrations for multi-stage language model programs. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 9340–9366, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.525. URL <https://aclanthology.org/2024.emnlp-main.525/>.
- Pryzant, R., Iter, D., Li, J., Lee, Y., Zhu, C., and Zeng, M. Automatic prompt optimization with “gradient descent” and beam search. In Bouamor, H., Pino, J., and Bali, K. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 7957–7968, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.494. URL <https://aclanthology.org/2023.emnlp-main.494/>.
- Singh, A., Fry, A., Perelman, A., Tart, A., Ganesh, A., El-Kishky, A., McLaughlin, A., Low, A., Ostrow, A., Ananthram, A., et al. Openai gpt-5 system card, 2025. URL <https://arxiv.org/abs/2601.03267>.

- Trivedi, H., Balasubramanian, N., Khot, T., and Sabharwal, A. Musique: Multihop questions via single-hop question composition, 2022. URL <https://arxiv.org/abs/2108.00573>.
- Wang, Y., Ma, X., Zhang, G., Ni, Y., Chandra, A., Guo, S., Ren, W., Arulraj, A., He, X., Jiang, Z., Li, T., Ku, M., Wang, K., Zhuang, A., Fan, R., Yue, X., and Chen, W. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark, 2024. URL <https://arxiv.org/abs/2406.01574>.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models, 2023. URL <https://arxiv.org/abs/2201.11903>.
- Yang, C., Wang, X., Lu, Y., Liu, H., Le, Q. V., Zhou, D., and Chen, X. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Bb4VGOWELI>.
- Yuksekgonul, M., Bianchi, F., Boen, J., Liu, S., Huang, Z., Guestrin, C., and Zou, J. Textgrad: Automatic "differentiation" via text, 2024. URL <https://arxiv.org/abs/2406.07496>.
- Zhong, W., Wang, S., Tang, D., Xu, Z., Guo, D., Wang, J., Yin, J., Zhou, M., and Duan, N. Ar-lsat: Investigating analytical reasoning of text, 2021. URL <https://arxiv.org/abs/2104.06598>.
- Zhou, Y., Muresanu, A. I., Han, Z., Paster, K., Pitis, S., Chan, H., and Ba, J. Large language models are human-level prompt engineers. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=92gvk82DE->.

Table 7. Effect of varying G , the number of top error categories selected for guidance generation, on final prompt accuracy for the AIME and FOLIO validation sets. Each dataset’s accuracy saturates at a different G , suggesting the optimal number of error categories varies by task.

G	AIME	FOLIO
Baseline	52.35 ± 0.91	72.50 ± 0.08
3	52.15 ± 0.50	78.35 ± 0.60
5	<u>52.83 ± 0.12</u>	79.72 ± 0.21
10	53.72 ± 0.28	<u>79.53 ± 0.53</u>

A. Experiment on number of top error categories

In this experiment, shown in Table 7, we analyze the effect of the parameter G , which determines the number of top error categories selected for guidance generation. For AIME, accuracy improves steadily as G increases, reflecting the presence of more than ten informative error categories. In contrast, performance on FOLIO saturates after $G = 5$. This behavior is likely due to the fact that FOLIO typically yields around ten error categories, and categories beyond the top five contribute limited additional signal for learning effectiveness.

B. Error Taxonomy Creation Prompts

Error taxonomy creation uses two prompts to categorize errors found in failed traces. The first prompt, shown in Figure 4 and Figure 5, categorizes the first batch of traces. The second prompt, shown in Figure 6 and Figure 7, also contains the error categories discovered so far and prompts the LLM to categorize into existing categories, only creating new categories if substantially different errors are encountered. Note that both prompts are split across figures due to their length.

C. Guidance Generation Prompt

Figure 8 and Figure 9 contain the prompt used for guidance generation. Again, note that the prompt is split across figures due to its length.

D. ETGPO-Optimized Prompts

We present prompts optimized by ETGPO for the datasets HMMT, MMLU-Pro, Musique, AR-LSAT, and FOLIO in Figure 10, Figure 11, Figure 12, Figure 13, and Figure 14 respectively.

```

You are an expert at analyzing why language models fail on {domain_description}.

## Failure 1
## Failure (Problem {problem_idx}, Run {run_id})

### Problem
{problem}

### Correct Answer
{correct_answer}

### Model's Reasoning
{reasoning}

### Model's Answer
{predicted_answer}

---

## Failure 2
## Failure (Problem {problem_idx}, Run {run_id})
...

## Your Task

Analyze each failure and identify the root cause of each error. Be as descriptive as possible.

For each failure, find:
1. The EARLIEST point in the reasoning where something went wrong
2. What specifically went wrong (calculation error, wrong approach, misunderstanding, etc.)
3. Why this error led to the wrong final answer

Create issue categories that capture each type of error. Categories should be general enough to potentially apply to other traces, but specific enough to be meaningful.

IMPORTANT: Each category must be SELF-CONTAINED and understandable by someone who has NOT seen the original problems.

## Output Format

Return a JSON object with:
```json
{
 "categories": [
 {
 "category_name": "Short descriptive name for this type of error",
 "summary": "One sentence describing the core error pattern.",
 "description": "Detailed description of what goes wrong in this category. Be very specific.",
 "example": "A concrete, self-contained example. Format: 'Problem: [simple problem]. Error: [what the model does wrong]. Correct: [what should happen].'",
 "error_type": "Type of error (e.g., Calculation Error, Wrong Approach, Conceptual Misunderstanding, Missing Step, Logical Fallacy, Factual Error, Incomplete Reasoning, Misreading the Problem)",
 "why_leads_to_wrong_answer": "Explanation of how this error causes wrong answers"
 }
],

```

Figure 4. Error taxonomy creation prompt for first batch (Part 1 of 2)



```

 "failure_assignments": [
 {
 "failure_id": 1,
 "problem_idx": <problem_idx>,
 "run_id": <run_id>,
 "category_name": "Name of the category this failure belongs to",
 "trace_details": {
 "trace_specific_location": "Where in the reasoning the error occurred",
 "trace_specific_details": "Specific details about what went wrong"
 }
 }
]
 }
 ...

```

Figure 5. Error taxonomy creation prompt for first batch (Part 2 of 2)

```

You are an expert at analyzing why language models fail on {domain_description}.

Existing Issue Categories

Category: {category_name}
- Summary: {summary}
- Description: {description}
- Example: {example}
- Error Type: {error_type}
- Why it leads to wrong answer: {why_leads_to_wrong_answer}
- Traces with this issue so far: {trace_count}

Category: {category_name}
...

Failure 1
Failure (Problem {problem_idx}, Run {run_id})

Problem
{problem}

Correct Answer
{correct_answer}

Model's Reasoning
{reasoning}

Model's Answer
{predicted_answer}

Failure 2
Failure (Problem {problem_idx}, Run {run_id})
...

Your Task

For each failure:
1. Determine if the error fits one of the EXISTING categories
2. OR create a NEW category if the error is fundamentally different

```

Figure 6. Error taxonomy creation prompt for subsequent batches (Part 1 of 2)

```

Output Format

Return a JSON object with:
```json
{
  "new_categories": [
    {
      "category_name": "Short descriptive name for NEW error type",
      "summary": "One sentence describing the core error pattern.",
      "description": "Detailed description of what goes wrong.",
      "example": "A concrete example.",
      "error_type": "Type of error (e.g., Calculation Error, Wrong Approach, Conceptual Misunderstanding, Missing Step, Logical Fallacy, Factual Error, Incomplete Reasoning, Misreading the Problem)",
      "why_leads_to_wrong_answer": "Explanation of how this error causes wrong answers"
    }
  ],
  "failure_assignments": [
    {
      "failure_id": 1,
      "problem_idx": <problem_idx>,
      "run_id": <run_id>,
      "is_new_category": false,
      "category_name": "Name of existing or new category",
      "trace_details": {
        "trace_specific_location": "Where in the reasoning the error occurred",
        "trace_specific_details": "Specific details about what went wrong"
      }
    }
  ]
}
```

Note: "new_categories" should only contain categories that don't exist yet.

```

Figure 7. Error taxonomy creation prompt for subsequent batches (Part 2 of 2)

```

You are an expert at improving language model performance on {domain_description}.

I have identified the following error categories from model failures. Generate guidance to help avoid these errors.

Category 1: {category_name}

Statistics: {failure_count} failures ({coverage_pct}%), {problem_count} problems

Summary: {summary}

Description: {description}

Example: {example}

Error Type: {error_type}

Why it leads to wrong answer: {why_leads_to_wrong_answer}

```

Figure 8. Guidance generation prompt (Part 1 of 2)

```
Category 2: {category_name}
...

Your Task

Generate guidance text that:
1. Addresses each failure category with specific, actionable advice
2. Is written as instructions TO the model
3. Uses concrete examples where helpful
4. Is prioritized by frequency

Generate DETAILED guidance with examples. Each item should include:
- Description of the error pattern
- Actionable advice on how to avoid it
- WRONG example showing the error
- CORRECT example showing proper approach

Critical Constraints

- The goal is ACCURACY, not caution. Never generate guidance that encourages the model
 to refuse, abstain, or say "not specified" when an answer can be reasonably
 provided.
- CORRECT examples must always show the model providing a substantive answer. Never
 show abstention/refusal as the correct behavior.

Output Format

Return a JSON object with:
```json
{
  "guidance_items": [
    {
      "category_name": "Name of the category",
      "guidance_text": "The full guidance text for this category"
    }
  ],
  "preamble": "1-2 sentence introduction",
  "full_prompt": "Complete enhanced prompt starting with base instruction"
}
```

The "full_prompt" should start with:
"Please think step by step and then solve the task."

Then add your preamble and guidance items.
```

Figure 9. Guidance generation prompt (Part 2 of 2)

Please think step by step and then solve the task.

To maximize accuracy and reliability on math problems, carefully follow these detailed instructions addressing common error patterns. Each guideline targets a specific failure mode and provides actionable steps and examples to help you avoid mistakes.

#### 1. Algebraic Manipulation Error in Multi-Variable Systems:

Description: Errors occur when manipulating or solving systems of equations with multiple variables, often due to misapplied algebraic identities, incorrect substitutions, or arithmetic mistakes. This leads to incorrect relationships and invalid solutions.

##### Actionable Advice:

- Carefully expand, simplify, and manipulate equations, especially when dealing with sums, products, or powers of variables.
- Explicitly write out algebraic identities (e.g.,  $(x + y)^2 = x^2 + 2xy + y^2$ ) and check each step for correctness.
- When substituting, ensure all variables and terms are correctly replaced and simplified.
- After solving, substitute solutions back into the original equations to verify correctness.

##### WRONG Example:

Given  $x + y = 5$  and  $x^2 + y^2 = 13$ , writing  $(x + y)^2 = x^2 + y^2 = 25$ , missing the cross term.

##### CORRECT Example:

$(x + y)^2 = x^2 + 2xy + y^2 = 25$ , so  $13 + 2xy = 25 \Rightarrow 2xy = 12 \Rightarrow xy = 6$ . Now solve  $x$  and  $y$  using  $x + y = 5$  and  $xy = 6$ .

#### 2. Failure to Recognize Structural Constraints:

Description: This error arises when key problem constraints (such as distinctness, divisibility, maximality/minimality) are overlooked or misapplied, leading to invalid or incomplete solutions.

##### Actionable Advice:

- Carefully identify and list all explicit and implicit constraints in the problem statement.
- For each candidate solution, verify that all constraints are satisfied before accepting it.
- When searching for maximal/minimal values, check all possibilities up to the specified bound.
- For problems involving properties like primality, distinctness, or divisibility, check each property for every element in the solution.

##### WRONG Example:

Find the largest integer  $n \leq 100$  such that  $n$  and  $n+1$  are both primes. Listing 97 and 98 as a pair without checking that 98 is not prime.

##### CORRECT Example:

Check each  $n \leq 100$ , verify both  $n$  and  $n+1$  are prime. The largest such  $n$  is 2 (since 2 and 3 are both prime).

#### 3. Failure to Complete Enumeration or Compute Final Numeric Answer:

Description: The model sets up a correct formula or summation but fails to compute the explicit final numeric answer, or plugs in an incorrect value.

...

Figure 10. ETGPO-optimized prompt for the HMMT dataset using GPT-4.1-mini (backbone) and GPT-4.1 (optimizer). Truncated.

Please think step by step and then solve the task.

To maximize accuracy on multiple choice reasoning questions, carefully follow the detailed guidance below for each common error pattern. These instructions will help you avoid frequent pitfalls and select the best answer in context.

1. Misidentification of Most Plausible or Expected Answer  
 Description: This error occurs when you select an answer that is factually supported by the passage or data, but fail to recognize that the question is asking for the most plausible, expected, or typical answer in a specific context (such as population-level, cultural, or comparative plausibility). You may default to the most directly supported or familiar fact, rather than considering which answer best fits the comparative or contextual framing of the question.

Actionable Advice:

- Always check if the question is asking for the 'most plausible', 'most likely', 'expected', or 'best' answer in a specific context (e.g., population, culture, or comparison).
- Compare all answer choices in light of the context and select the one that best fits the comparative or expected framing, not just one that is factually supported.
- Ask yourself: "Which answer would be most typical or expected in this context, even if others are also supported?"

WRONG Example:  
 Passage: Discusses health benefits of sauna use in Finland.  
 Question: Which outcome is most plausible in Finland?  
 Model selects: 'Lower rates of cardiovascular-related deaths among frequent sauna users' (factually supported, but not the most contextually plausible answer).

CORRECT Example:  
 Model selects: 'An 86-year old male mayor who is revered in the community' (more plausible as a population-level or cultural outcome in Finland, given the context).

2. Procedural Error in Multi-Step Quantitative Reasoning  
 Description: This error happens when you make a mistake in one or more steps of a multi-step quantitative problem, such as misapplying formulas, failing to account for constraints (like material widths or wastage), or misinterpreting which dimensions or units to use. Errors in any step can propagate and lead to an incorrect final answer.

Actionable Advice:

- Break down the problem into clear, sequential steps and explicitly state each calculation.
- Carefully check the requirements and constraints (e.g., roll width, rounding up to whole units, wastage).
- After each step, verify that your result makes sense before proceeding.
- Double-check unit conversions and ensure you are using the correct dimensions/formulas for each step.

WRONG Example:  
 Problem: Calculate the cost of carpeting a room using rolls of fixed width.  
 Model calculates: Total area of the room and multiplies by cost per area, ignoring roll width and necessary strips.

CORRECT Example:  
 Model calculates: Number of strips needed based on roll width and room dimensions, then total area of carpet required, then multiplies by cost per area, accounting for all constraints.

...

Figure 11. ETGPO-optimized prompt for the MMLU-Pro dataset using GPT-4.1-mini (backbone) and GPT-4.1 (optimizer). Truncated.



Please think step by step and then solve the task.

To maximize accuracy on multi-hop reasoning questions, carefully follow these detailed instructions for each common error type. Applying these strategies will help ensure your answers are precise, fully supported, and contextually appropriate.

#### 1. Failure to Identify the Most Specific or Relevant Answer

Description: This error occurs when you provide a general or partially correct answer instead of the most specific or directly relevant one required by the question. Often, you may stop at a higher-level category or a more general answer, missing a more precise or contextually appropriate response.

Actionable Advice:

- Always check if the question asks for a specific form of the answer (e.g., symbol, date, name, location).
- If multiple levels of specificity are possible, select the most precise one that directly answers the question.
- Re-read the question to identify keywords like 'symbol', 'exact', 'full name', or 'specific'.

WRONG Example:

Question: 'What is the chemical symbol for the element with atomic number 8?'

Answer: 'Oxygen.'

CORRECT Example:

Answer: 'O.'

WRONG Example:

Question: 'Which district is the city of Cambridge located in?'

Answer: 'Cambridgeshire.' (This is the county, not the district.)

CORRECT Example:

Answer: 'Cambridge district.'

#### 2. Failure to Correctly Chain Multi-Hop Reasoning Steps

Description: This error happens when you skip a necessary reasoning step, make an incorrect connection, or fail to maintain the correct chain of logic in multi-hop questions. This often leads to answers that are plausible based on partial information, but not fully supported by the required reasoning chain.

Actionable Advice:

- Break down the question into individual reasoning steps and solve each step in order.
- Explicitly identify intermediate entities or facts before proceeding to the next step.
- Do not stop after the first plausible answer; ensure all steps are completed.

WRONG Example:

Question: 'Who was the president of the country whose capital is Paris in 2010?'

Answer: 'Emmanuel Macron.' (Current president, not the one in 2010.)

CORRECT Example:

Step 1: The capital Paris is in France.

Step 2: Who was president of France in 2010? Nicolas Sarkozy.

Answer: 'Nicolas Sarkozy.'

...

Figure 12. ETGPO-optimized prompt for the Musique dataset using GPT-4.1-mini (backbone) and GPT-4.1 (optimizer). Truncated.

Please think step by step and then solve the task.

To maximize logical reasoning accuracy, carefully follow these detailed instructions to avoid common error patterns. Each guideline addresses a specific reasoning pitfall and provides concrete steps and examples to ensure correct answer selection.

1. Failure to Systematically Enumerate Possibilities:

- Always systematically list or consider every possible arrangement or scenario that fits the given constraints before making deductions.
- Use tables, lists, or diagrams to track possibilities and eliminate options only after thorough checking.
- WRONG: Only checking a few arrangements and missing valid ones.
- CORRECT: Listing all possible assignments and checking each against the constraints.

2. Constraint Overlook or Misapplication:

- Carefully identify and restate every constraint before beginning your reasoning.
- At each step, check that all constraints are being enforced for every configuration you consider.
- WRONG: Allowing arrangements that violate a stated constraint.
- CORRECT: Explicitly checking each arrangement for all constraints.

3. Overgeneration of Necessary Conclusions:

- For 'must be true' questions, only select options that are true in every possible valid configuration, not just in some.
- Test each answer choice by considering whether it could ever be false under the given constraints.
- WRONG: Selecting multiple 'must be true' answers when only one is required.
- CORRECT: Only selecting the answer that is necessarily true in all cases.

4. Misapplication of 'Must Be True' vs. 'Could Be True':

- For 'must be true' questions, verify that the statement holds in every possible configuration allowed by the constraints.
- For 'could be true' questions, check if there is at least one valid configuration where the statement holds.
- WRONG: Claiming something 'must be true' just because it is possible in some cases.
- CORRECT: Distinguishing between necessity and possibility by checking all scenarios.

5. Misinterpretation of 'Cannot' or 'Except' Questions:

- For 'cannot be true' questions, look for the answer choice that is impossible under the constraints.
- For 'except' questions, identify the one option that does not fit the stated pattern.
- Restate the question in your own words to clarify what is being asked.
- WRONG: Selecting an option that is possible when asked for something impossible.
- CORRECT: Checking each answer for impossibility and selecting accordingly.

6. Failure to Recognize Mutual Exclusivity or Uniqueness:

- For assignments, ensure that each entity occupies only one position, and each position is filled by only one entity, unless stated otherwise.
- Explicitly check for overlaps or double assignments in your reasoning.
- WRONG: Assigning two entities to the same unique position.
- CORRECT: Ensuring each assignment is unique and non-overlapping.

Figure 13. ETGPO-optimized prompt for the AR-LSAT dataset using GPT-4.1-mini (backbone) and GPT-4.1 (optimizer).

Please think step by step and then solve the task.

To improve logical reasoning accuracy, carefully distinguish between what is entailed, contradicted, or left uncertain by the premises. Use the following detailed guidance to avoid common reasoning errors.

1. Confusing 'False' with 'Uncertain' in Logical Reasoning  
 Description: This error occurs when you treat a conclusion that is not entailed by the premises as definitively false, rather than recognizing that it is simply uncertain given the information provided. If the premises do not guarantee the truth or falsity of the conclusion, the correct answer is 'Uncertain.'

Actionable Advice:

- Do NOT assume that a conclusion is false just because it cannot be proven true from the premises.
- Only answer 'False' if the conclusion is directly contradicted by the premises.
- If the premises are compatible with both the conclusion and its negation, answer 'Uncertain.'

WRONG Example:  
 Premises: Some cats are black.  
 Conclusion: All cats are black.  
 Model's (incorrect) answer: False (because the premises don't prove the conclusion)

CORRECT Example:  
 Premises: Some cats are black.  
 Conclusion: All cats are black.  
 Proper reasoning: The premises do not guarantee or rule out the conclusion. The answer is 'Uncertain.'

2. Mistaking Direct Contradiction for Uncertainty  
 Description: This error happens when you fail to recognize that a conclusion directly contradicts a premise, and instead label it as 'Uncertain.' If a conclusion is explicitly ruled out by the premises, it must be marked as 'False.'

Actionable Advice:

- Always check if the conclusion is the direct negation of any premise.
- If the conclusion is incompatible with the premises, answer 'False.'
- Do NOT answer 'Uncertain' when a contradiction exists.

WRONG Example:  
 Premises: All dogs bark.  
 Conclusion: Some dogs do not bark.  
 Model's (incorrect) answer: Uncertain

CORRECT Example:  
 Premises: All dogs bark.  
 Conclusion: Some dogs do not bark.  
 Proper reasoning: The conclusion directly contradicts the premise. The answer is 'False.'

3. Illicit Elimination of Logical Possibilities  
 Description: This error arises when you incorrectly eliminate possible cases, treating a conclusion as false when it is merely not entailed by the premises. If the premises do not rule out a conclusion, but also do not entail it, the answer should be 'Uncertain.'

Actionable Advice:

- Do NOT treat the absence of evidence as evidence of falsity.

...

Figure 14. ETGPO-optimized prompt for the FOLIO dataset using GPT-4.1-mini (backbone) and GPT-4.1 (optimizer). Truncated.